

Demonstration of SpatIO: Spatial Physical Computing Toolkit Based on Extended Reality

Seung Hyeon Han
Department of Industrial Design
KAIST
Daejeon, Republic of Korea
hsh0908y@kaist.ac.kr

Yeeun Han
Department of Industrial Design,
KAIST
Daejeon, Republic of Korea
yeeunhan@kaist.ac.kr

Kyeongho Park
Department of Industrial design
KAIST
Daejeon, Republic of Korea
rudghqnr@kaist.ac.kr

Sangjun Lee
Department of Industrial Design
KAIST
Daejeon, Republic of Korea
sangjunlee@kaist.ac.kr

Woohun Lee
Department of Industrial Design
KAIST
Daejeon, Republic of Korea
woohun.lee@kaist.ac.kr

Abstract

Current physical computing tools do not effectively support the placement of sensors and actuators in three-dimensional space, despite its importance in designing spatial and proxemic interactions. To address this, we present SpatIO, an XR-based physical computing toolkit that supports a continuous end-to-end workflow. SpatIO consists of three interconnected subsystems: *SpatIO Environment* for composing and testing prototypes with virtual sensors and actuators, *SpatIO Module* for converting virtually placed components into physical ones, and *SpatIO Code* for authoring interactions with spatial visualization of data flow.

CCS Concepts

• **Human-centered computing** → **Interaction design process and methods; Interaction design theory, concepts and paradigms; Virtual reality.**

Keywords

Spatial Physical Computing, Prototyping, Extended Reality, Digital Fabrication, Visual Programming

ACM Reference Format:

Seung Hyeon Han, Yeeun Han, Kyeongho Park, Sangjun Lee, and Woohun Lee. 2025. Demonstration of SpatIO: Spatial Physical Computing Toolkit Based on Extended Reality. In *The 38th Annual ACM Symposium on User Interface Software and Technology (UIST Adjunct '25)*, September 28–October 01, 2025, Busan, Republic of Korea. ACM, New York, NY, USA, 3 pages. <https://doi.org/10.1145/3746058.3759014>

1 Introduction

When designing spatial and proxemic interactions, the placement of sensors and actuators is crucial. To capture spatial data—such as

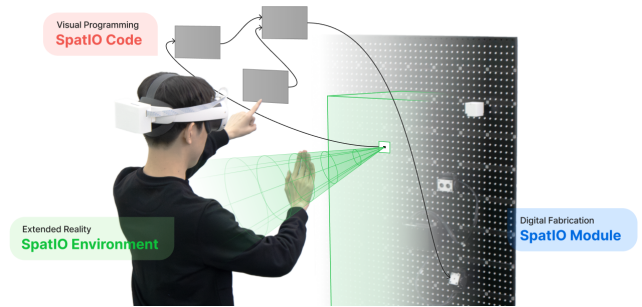


Figure 1: The concept of SpatIO and its subcomponents.

user location and orientation [1, 4]—in an open space, sensors must be strategically positioned to fully cover the relevant areas.

However, placing components in physical space is challenging. Determining the optimal placement of sensors is difficult because their detection range is invisible. Mounting them at the desired location and orientation introduces further complexity. Despite the importance of sensor *placement*, especially for spatial data [3], this aspect has received relatively little attention.

One promising approach is to leverage extended reality (XR). Virtual reality allows designers to build and test prototypes without needing physical materials. Augmented reality helps in constructing physical prototypes by overlaying guidance in 3D space.

However, this introduces another challenge: how can we transition virtual prototypes into physical ones? Although many research works explore utilizing virtual reality for physical computing, most treat virtual and physical prototyping separately [2, 6].

In this demonstration, we showcase *SpatIO* [2], an XR-based physical computing toolkit that supports a continuous end-to-end workflow. SpatIO allows designers to create virtual prototypes and incrementally transition them into physical ones.

2 From Virtual to Physical Prototype

To support a seamless transition from virtual to physical prototypes, we structured the workflow of SpatIO into three phases, inspired by the concept of the reality-virtuality continuum [5]:

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).

UIST Adjunct '25, Busan, Republic of Korea

© 2025 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2036-9/25/09

<https://doi.org/10.1145/3746058.3759014>

- *Virtual phase*: The designer builds a fully virtual prototype by iteratively placing, programming, and testing virtual components.
- *Mixed phase*: The designer gradually replaces virtual components with their physical counterparts, working with a hybrid of virtual and real components.
- *Real phase*: Once all components are replaced with physical ones, the designer removes the headset and interacts with the fully functional prototype independent of the virtual environment.

3 SpatIO

SpatIO consists of three subsystems: *SpatIO Environment*, *SpatIO Module*, and *SpatIO Code*. Together, they bridge physical computing with three different areas of human-computer interaction (HCI): extended reality, digital fabrication, and visual programming.

3.1 SpatIO Environment

The *SpatIO Environment* is an extended reality environment where designers can build and test prototypes. The environment is implemented to run on the Meta Quest 3 using Unity and Meta XR SDKs.

Designers begin by customizing a virtual product's size and shape to define its external structure. Then, virtual sensors and actuators can be added to the prototype's surface. They can freely move and rotate these components to configure their proper placements. One unique thing in SpatIO is that the location and orientation are snapped to a discrete grid, simplifying future transitions to physical placement.

After placement, SpatIO simulates each component's behavior. For sensors, particularly those detecting human presence, SpatIO uses body-tracking data from the headset. With this, SpatIO simulates four key sensor types frequently used in spatial interactions: distance sensors, motion sensors, color cameras, and depth cameras.

SpatIO also visualizes input and output data, including three-dimensional representations of sensor ranges and values. Unlike prior works that used solid shapes or plain surfaces, SpatIO adds radial and concentric wireframe lines to improve perceptibility—especially when viewed from inside the sensor range in immersive environments.

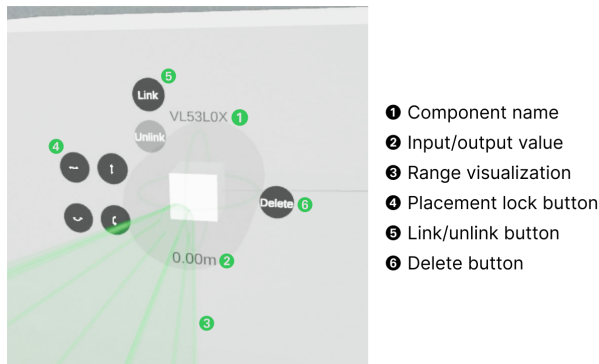


Figure 2: A virtual module in SpatIO Environment.

3.2 SpatIO Module

Once the virtual prototype is ready, designers enter the mixed phase, where virtual components are gradually substituted with physical ones. The *SpatIO Module* system enables this by helping designers place physical components at the exact same position as their virtual counterparts.

The *IO Module* is a small cuboid-shaped module that houses a sensor or actuator and a custom-designed IO Board. The *Hub Module* is a slightly larger cuboid containing a Raspberry Pi along with a custom Hub Board, a DFR0494 UPS (uninterrupted power supply) module, and a 3500mAh lithium-ion polymer battery. When an IO Module connects to the Hub Module, it sends the data to the headset via WiFi. SpatIO Environment then automatically switches from using virtual data to real IO data.

To support the physical placement of the modules, we introduce a novel approach of pegboard-based positioning. *Attachment Modules* have bumps that can be inserted into the holes in pegboards. By combining Attachment Modules at the back of IO and Hub Modules, designers can place them at the desired position on the pegboards.

While this offers an easy method for positioning components in two-dimensional space, it only supports two degrees of freedom (DOF) of positioning. To support full 6-DOF placements, we designed the *Positioner Module*, which can be inserted between an IO Module and an Attachment module. We developed multiple different types of Positioner Modules, such as XY-Rotation Module, Z-Rotation Module, and X-, Y-, and Z-Extension Module. Positioner Modules can also be daisy-chained to allow more complex configurations.



Figure 3: The SpatIO Module system. The combined modules can be inserted into a pegboard.

Although Positioner and Attachment Modules enable easy and precise *initial* physical placements matching virtual placements, the system will lose track when the designer changes placement after that. To track these changes, we added four small ARUco

markers at the front face of each IO Module. We also developed *Vision Module* that can be mounted on the headset to capture these markers and send them to the SpatIO Environment.

3.3 SpatIO Code

We explored various programming paradigms for SpatIO and chose flow-based (or node-based) programming due to its alignment with the concept of SpatIO: spatiality. It allows each node to be freely placed. Also, its graph-like structure with nodes and edges themselves graphically represents the flow of the data.

Unlike traditional screen-based flow editors, SpatIO allows nodes to be placed anywhere in 3D space. This enables spatial correlations between code and hardware components, fostering more intuitive programming.

Another key feature of SpatIO Code is dynamic execution across all phases. When a component is physically connected or disconnected in the mixed phase, SpatIO Code switches between virtual and real IO data. Once all components are physically placed, the code can now run independently from the virtual environment—allowing the designer to remove the headset and enter the real phase. In real phase, the Hub Module executes the code without the need to be connected to the headset.

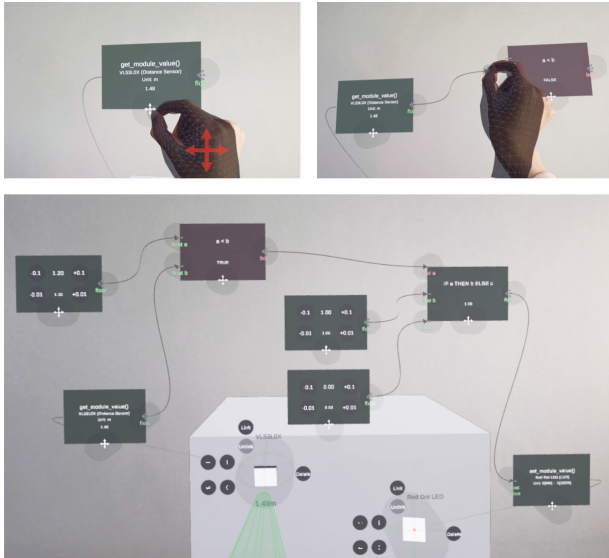


Figure 4: Nodes in SpatIO Code. They can be freely moved in three-dimensional space and connected to another node.

4 Conclusion

Now we have walked through all of the constituents of the SpatIO system. SpatIO allows designers to easily build spatial configurations of sensors and actuators in a virtual environment and helps them gradually convert these configurations into a physical prototype. Reflecting on this system, we propose the concept of *spatial physical computing*, which emphasizes spatial work, coherent workspace, and continuous workflow [2].

Acknowledgments

This work was supported by the IITP (Institute of Information & Communications Technology Planning & Evaluation) - ITRC (Information Technology Research Center) grant funded by the Korea government (Ministry of Science and ICT) (IITP-2025-RS-2024-00436398).

References

- [1] Till Ballendat, Nicolai Marquardt, and Saul Greenberg. 2010. Proxemic interaction: designing for a proximity and orientation-aware environment. In *ACM International Conference on Interactive Tabletops and Surfaces*. 121–130.
- [2] Seung Hyeon Han, Yeeun Han, Kyeongho Park, Sangjun Lee, and Woohun Lee. 2025. SpatIO: Spatial Physical Computing Toolkit Based on Extended Reality. In *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems*. 1–22.
- [3] Yoonji Kim, Junyi Zhu, Mihir Trivedi, Dishita Turakhia, Ngai Hang Wu, Donghyeon Ko, Michael Wessely, and Stefanie Mueller. 2022. SensorViz: Visualizing Sensor Data Across Different Stages of Prototyping Interactive Objects. In *Proceedings of the 2022 ACM Designing Interactive Systems Conference*. 987–1001.
- [4] Nicolai Marquardt, Robert Diaz-Marino, Sebastian Boring, and Saul Greenberg. 2011. The proximity toolkit: prototyping proxemic interactions in ubiquitous computing ecologies. In *Proceedings of the 24th annual ACM symposium on User interface software and technology*. 315–326.
- [5] Paul Milgram, Haruo Takemura, Akira Utsumi, and Fumio Kishino. 1995. Augmented reality: A class of displays on the reality-virtuality continuum. In *Telemanipulator and telepresence technologies*, Vol. 2351. Spie, 282–292.
- [6] Hongbo Zhang, Pei Chen, Xuelong Xie, Zhaoqu Jiang, Yifei Wu, Zejian Li, Xiaoyu Chen, and Lingyun Sun. 2025. FusionProtor: A Mixed-Prototype Tool for Component-level Physical-to-Virtual 3D Transition and Simulation. In *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems*. 1–19.